

Problem Set 1

Applied Stats II

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the i th ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnov CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an R function that implements this test where the reference distribution is normal. Using R generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
1 # create empirical distribution of observed data
2 ECDF <- ecdf(data)
3 empiricalCDF <- ECDF(data)
4 # generate test statistic
5 D <- max(abs(empiricalCDF - pnorm(data)))
```

Answer:

I generated 1,000 random variables from a Cauchy distribution and calculated the KS statistic D . Then, I implemented the p-value approximation using the provided infinite series formula.

```
1
2 # [Task] Generate 1000 Cauchy random variables
3 # NOTE: Defined 'data' here so the template code below runs correctly
4 data <- rcauchy(1000, location = 0, scale = 1)
5
6 # create empirical distribution of observed data
7 ECDF <- ecdf(data)
8 empiricalCDF <- ECDF(data)
9 # generate test statistic
10 D <- max(abs(empiricalCDF - pnorm(data)))
11
12 print(paste("KS Statistic D:", D))
13
14 # [Task] Calculate p-value using the infinite series formula
15 calculate_ks_pvalue <- function(d) {
16   sum_term <- 0
17   # Iterate 1000 times for approximation
18   for(k in 1:1000) {
19     term <- exp(-((2*k - 1)^2 * pi^2) / (8 * d^2))
20     sum_term <- sum_term + term
21
22     # Check convergence
23     if(term < 1e-15) break
24   }
25   p_val <- (sqrt(2 * pi) / d) * sum_term
26   return(p_val)
27 }
28
29 p_value <- calculate_ks_pvalue(D)
```

Results:

- **KS Statistic (D):** 0.1347281
- **Calculated p-value:** 5.65×10^{-29}

Since the p-value is effectively 0, we reject the null hypothesis that the data comes from a Normal distribution. Substantively, this result confirms that the data generating process (Cauchy) produces extreme values (heavy tails) that are fundamentally distinct from the Gaussian assumption used in the test.

Question 2

Estimate an OLS regression in R that uses the Newton-Raphson algorithm (specifically BFGS, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

```
1 set.seed (123)
2 data <- data.frame(x = runif(200, 1, 10))
3 data$y <- 0 + 2.75*data$x + rnorm(200, 0, 1.5)
4
```

Answer:

I defined a loss function to calculate the Sum of Squared Residuals (SSR) and minimized it using the `optim()` function with the BFGS method. The data was generated with a true intercept of 0 and a slope of 2.75.

```
1 set.seed (123)
2 data <- data.frame(x = runif(200, 1, 10))
3 data$y <- 0 + 2.75*data$x + rnorm(200, 0, 1.5)
4
5 # [Task] Estimate OLS using BFGS (optim) vs lm
6
7 # 1. Define Loss Function (Sum of Squared Residuals)
8 ols_loss_fn <- function(theta, y, x) {
9   beta0 <- theta[1]
10  beta1 <- theta[2]
11
12  # Linear model prediction
13  y_pred <- beta0 + beta1 * x
14
15  # Sum of Squared Errors
16  sum((y - y_pred)^2)
17 }
18
19 # 2. Run Optimization using BFGS
20 # Initial guess (Intercept=0, Slope=0)
21 initial_params <- c(0, 0)
22
23 opt_res <- optim(par = initial_params,
24                 fn = ols_loss_fn,
25                 y = data$y,
26                 x = data$x,
27                 method = "BFGS")
28
29 print("--- BFGS Results ---")
30 print(opt_res$par)
31
32 # 3. Compare with standard lm()
33 lm_res <- lm(y ~ x, data = data)
```

Results: Comparing the coefficients, the BFGS algorithm yields results nearly identical to the standard `lm` function:

```
--- BFGS Results ---  
[1] 0.1391778  2.7267000  
  
--- lm Results ---  
(Intercept)          x  
0.1391874          2.7266985
```

Interpretation: Substantively, the estimated slope coefficient of approximately **2.73** implies that a one-unit increase in x is associated with a 2.73-unit increase in y . This estimate aligns closely with the true parameter (2.75) used in the data generation process.

The difference between the optimization method and the analytical OLS solution is negligible (1.11×10^{-5}). This confirms that the BFGS algorithm successfully minimized the unexplained variation (residuals) to the same level as the standard linear model.